

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»
Дисциплина «Основы программной инженерии»

Лабораторная работа №3
Вариант №982735

Выполнил:
Пивоваров Р.Н.

Проверил:
Кобик Н.А.

Дата сдачи: 13 Июня
Санкт-Петербург, 2026

Содержание

1. Задание	3
2. сценарий ant	4
3. Заключение	20

1. Задание

Написать сценарий для утилиты Apache Ant, реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из лабораторной работы №3 по дисциплине «Веб-программирование».

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске класса.

Сценарий должен реализовывать следующие цели (targets):

- `compile` – компиляция исходных кодов проекта.
- `build` – компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели `compile`.
- `clean` – удаление скомпилированных классов проекта и всех временных файлов (если они есть).
- `test` – запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель `build`).
- `music` - воспроизведение музыки по завершению сборки (цель `build`).
- `doc` - добавление в MANIFEST.MF MD5 и SHA-1 файлов проекта, а также генерация и добавление в архив javadoc по всем классам проекта.
- `xml` - валидация всех xml-файлов в проекте.
- `scp` - перемещение собранного проекта по scp на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель `build`).
- `native2ascii` - преобразование `native2ascii` для копий файлов локализации (для тестирования сценария все строковые параметры необходимо вынести из классов в файлы локализации).
- `alt` - создаёт альтернативную версию программы с измененными именами переменных и классов (используя задание `replace/replaceregexp` в файлах параметров) и упаковывает её в jar-архив. Для создания jar-архива использует цель `build`.

- report - в случае успешного прохождения тестов сохраняет отчет junit в формате xml, добавляет его в репозиторий git и выполняет commit.
- env - осуществляет сборку и запуск программы в альтернативных окружениях; окружение задается версией java и набором аргументов виртуальной машины в файле параметров.
- diff - осуществляет проверку состояния рабочей копии, и, если изменения не касаются классов, указанных в файле параметров выполняет commit в репозиторий svn.
- history - если проект не удаётся скомпилировать (цель compile), загружается предыдущая версия из репозитория svn. Операция повторяется до тех пор, пока проект не удастся собрать, либо не будет получена самая первая ревизия из репозитория. Если такая ревизия найдена, то формируется файл, содержащий результат операции diff для всех файлов, изменённых в ревизии, следующей непосредственно за последней работающей.
- team - осуществляет получение из git-репозитория 4 предыдущих ревизий, их сборку (по аналогии с основной) и упаковку получившихся jar-файлов в zip-архив. Сборку реализовать посредством вызова цели build.

2. сценарий ant

build.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="WebLab3" default="build" basedir=".">

    <!-- Загрузка параметров из внешнего файла -->
    <property file="build.properties"/>

    <!-- Путь к библиотекам для компиляции -->
    <path id="compile.classpath">
        <fileset dir="${lib.compile}" includes="**/*.jar"
erroronmissingdir="false"/>
    </path>

    <!-- Путь к библиотекам для тестов -->
    <path id="test.classpath">
```

```

    <path refid="compile.classpath"/>
    <pathelement location="${dir.classes}"/>
    <pathelement location="${dir.test.classes}"/>
    <fileset dir="${lib.runtime}" includes="**/*.jar"
erroronmissingdir="false"/>
    <fileset dir="${lib.test}" includes="**/*.jar"
erroronmissingdir="false"/>
    </path>

    <!-- Подключаем ant-contrib для trycatch -->
    <taskdef resource="net/sf/antcontrib/antlib.xml">
        <classpath>
            <pathelement location="${ant.home}/lib/ant-
contrib-1.0b3.jar"/>
        </classpath>
    </taskdef>

    <!-- compile — компиляция исходных кодов -->
    <target name="compile" description="Компиляция исходных кодов
проекта">
        <mkdir dir="${dir.classes}"/>
        <javac
            srcdir="${dir.src}"
            destdir="${dir.classes}"
            source="${javac.source}"
            target="${javac.target}"
            encoding="${javac.encoding}"
            debug="${javac.debug}"
            includeantruntime="false">
            <classpath refid="compile.classpath"/>
        </javac>
        <!-- Копируем ресурсы -->
        <copy todir="${dir.classes}">
            <fileset dir="${dir.resources}" includes="**/*"
erroronmissingdir="false"/>
        </copy>
        <echo message="[compile] Компиляция завершена успешно."/>
    </target>

    <!-- build — сборка WAR-архива -->
    <target name="build" depends="compile" description="Сборка WAR-
архива">
        <mkdir dir="${dir.dist}"/>

```

```

<!-- Формируем MANIFEST.MF -->
<manifest file="${dir.build}/MANIFEST.MF">
    <attribute name="Manifest-Version" value="1.0"/>
    <attribute name="Implementation-Title"
value="${manifest.title}"/>
    <attribute name="Implementation-Version"
value="${manifest.version}"/>
    <attribute name="Implementation-Vendor"
value="${manifest.vendor}"/>
    <attribute name="Built-By"
value="${user.name}"/>
    <attribute name="Build-Jdk"
value="${java.version}"/>
</manifest>

<!-- Собираем WAR -->
<war destfile="${war.path}" webxml="${dir.webapp}/WEB-INF/
web.xml"
    manifest="${dir.build}/MANIFEST.MF">
    <fileset dir="${dir.webapp}">
        <exclude name="WEB-INF/web.xml"/>
    </fileset>
    <classes dir="${dir.classes}"/>
    <lib dir="${lib.runtime}" erroronmissingdir="false">
        <include name="**/*.jar"/>
    </lib>
    <metainf dir="${dir.resources}/META-INF"
erroronmissingdir="false"/>
</war>

    <echo message="[build] WAR собран: ${war.path}"/>
    <antcall target="music"/>
</target>

<!-- clean – удаление скомпилированных классов и временных
файлов -->
<target name="clean" description="Удаление скомпилированных
классов и временных файлов">
    <delete dir="${dir.build}"/>
    <delete dir="${dir.dist}"/>
    <delete dir="${dir.alt}"/>
    <delete dir="test-reports"/>
    <echo message="[clean] Временные файлы удалены."/>
</target>

```

```

<!--
    test — запуск JUnit-тестов
-->
<target name="test" depends="build" description="Запуск JUnit-
тестов">
    <mkdir dir="${dir.test.classes}"/>
    <mkdir dir="${dir.test.reports}"/>

    <!-- Компилируем тесты -->
    <javac
        srcdir="${dir.test.src}"
        destdir="${dir.test.classes}"
        source="${javac.source}"
        target="${javac.target}"
        encoding="${javac.encoding}"
        includeantruntime="false">
        <classpath refid="test.classpath"/>
    </javac>

    <!-- Запускаем тесты -->
    <junit printsummary="yes"
        haltonerror="${junit.haltonerror}"
        haltonfailure="${junit.haltonfailure}"
        fork="${junit.fork}">
        <classpath refid="test.classpath"/>
        <formatter type="xml"/>
        <batchtest todir="${dir.test.reports}">
            <fileset dir="${dir.test.classes}" includes="**/
*Test.class"/>
        </batchtest>
    </junit>

    <echo message="[test] Тесты завершены. Отчёты:
${dir.test.reports}"/>
</target>

<!--
    music — воспроизведение музыки по завершению сборки
-->
<target name="music" description="Воспроизведение музыки по
завершению сборки">
    <property name="music.path" location="${basedir}/
${music.file}"/>

```

```

    <available file="${music.path}" property="music.exists"/>
    <echo message="[music] Файл: ${music.path} существует:
${music.exists}"/>
    <antcall target="music.play"/>
</target>

<target name="music.play" if="music.exists">
    <exec executable="powershell" failonerror="false">
        <arg value="-NonInteractive"/>
        <arg value="-c"/>
        <arg value="Add-Type -AssemblyName System.Windows.Forms;
(New-Object System.Media.SoundPlayer '${music.path}').PlaySync()"/>
    </exec>
    <echo message="[music] Воспроизведение: ${music.file}"/>
</target>

<!--
    doc — генерация Javadoc + MD5/SHA-1 в MANIFEST.MF
-->
<target name="doc" depends="build" description="Генерация
Javadoc и добавление хешей в манифест">
    <mkdir dir="${dir.javadoc}"/>

    <!-- Генерируем Javadoc -->
    <javadoc
        sourcepath="${dir.src}"
        destdir="${dir.javadoc}"
        encoding="${javac.encoding}"
        docencoding="UTF-8"
        charset="UTF-8"
        author="true"
        version="true"
        windowtitle="${project.name} ${project.version} API">
        <classpath refid="compile.classpath"/>
        <packageset dir="${dir.src}" defaultexcludes="yes"/>
    </javadoc>

    <!-- Вычисляем MD5 и SHA-1 WAR-файла -->
    <checksum file="${war.path}" algorithm="MD5"
property="war.md5"/>
    <checksum file="${war.path}" algorithm="SHA-1"
property="war.sha1"/>

    <!-- Добавляем хеши в манифест и переупаковываем WAR -->

```

```

    <manifest file="${dir.build}/MANIFEST.MF" mode="update">
        <attribute name="WAR-MD5" value="${war.md5}"/>
        <attribute name="WAR-SHA1" value="${war.sha1}"/>
    </manifest>

    <!-- Добавляем Javadoc и обновлённый манифест в WAR -->
    <zip destfile="${war.path}" update="true">
        <zipfileset dir="${dir.javadoc}" prefix="javadoc"/>
        <zipfileset file="${dir.build}/MANIFEST.MF"
fullpath="META-INF/MANIFEST.MF"/>
    </zip>

    <echo message="[doc] Javadoc сгенерирован. MD5=${war.md5}
SHA1=${war.sha1}"/>
</target>

<!-- xml – валидация всех XML-файлов проекта -->
<target name="xml" description="Валидация XML-файлов проекта">
<xmlvalidate failonerror="true" lenient="true" warn="true">
    <fileset dir="${dir.webapp}">
        <include name="**/*.xml"/>
    </fileset>
    <fileset dir="${dir.resources}">
        <include name="**/*.xml"/>
    </fileset>
</xmlvalidate>
<echo message="[xml] Все XML-файлы прошли валидацию."/>
</target>

<!--
    scp – копирование WAR на удалённый сервер (вызывает build)
-->
<target name="scp" depends="build" description="Копирование WAR
на сервер по SCP">
<scp
    file="${war.path}"
    todir="${scp.user}@${scp.host}:${scp.remotedir}"
    password="${scp.password}"
    port="${scp.port}"
    trust="true"/>
    <echo message="[scp] WAR отправлен на ${scp.host}:
${scp.remotedir}"/>
</target>

```

```

<!-- native2ascii — конвертация файлов локализации -->
<target name="native2ascii" description="Конвертация файлов
локализации в ASCII">
  <mkdir dir="${dir.il8n.ascii}"/>
  <native2ascii
    encoding="UTF-8"
    src="${dir.il8n}"
    dest="${dir.il8n.ascii}"
    includes="**/*.properties"/>
  <echo message="[native2ascii] Файлы локализации сконвертированы
в: ${dir.il8n.ascii}"/>
</target>

<!--
  alt — альтернативная версия с заменой имён классов/
переменных
-->
<target name="alt" description="Создание альтернативной версии
с заменёнными именами">
  <mkdir dir="${dir.alt.src}"/>

  <!-- Копируем исходники в alt/src -->
  <copy todir="${dir.alt.src}">
    <fileset dir="${dir.src}" includes="**/*.java"/>
    <filterchain>
      <tokenfilter>
        <filetokenizer/>
      </tokenfilter>
    </filterchain>
  </copy>

  <!-- Заменяем имя класса -->
  <replaceregexp
    match="${alt.replace.from.class}"
    replace="${alt.replace.to.class}"
    flags="g"
    byline="false">
    <fileset dir="${dir.alt.src}" includes="**/*.java"/>
  </replaceregexp>

  <!-- Заменяем имя переменной -->
  <replaceregexp
    match="${alt.replace.from.var}"
    replace="${alt.replace.to.var}"

```

```

        flags="g"
        byline="false">
        <fileset dir="${dir.alt.src}" includes="**/*.java"/>
    </replaceregexp>

    <move file="${dir.alt.src}/app/backend/bean/InputBean.java"
        tofile="${dir.alt.src}/app/backend/bean/FormBean.java"/>

    <!-- Компилируем alt-версию -->
    <mkdir dir="${dir.alt.classes}"/>
    <javac
        srcdir="${dir.alt.src}"
        destdir="${dir.alt.classes}"
        source="${javac.source}"
        target="${javac.target}"
        encoding="${javac.encoding}"
        includeantruntime="false">
        <classpath refid="compile.classpath"/>
    </javac>

    <!-- Собираем alt WAR -->
    <mkdir dir="${dir.alt.dist}"/>
    <war destfile="${war.alt.path}" webxml="${dir.webapp}/WEB-INF/web.xml">
        <fileset dir="${dir.webapp}">
            <exclude name="WEB-INF/web.xml"/>
        </fileset>
        <classes dir="${dir.alt.classes}"/>
        <lib dir="${lib.runtime}" erroronmissingdir="false">
            <include name="**/*.jar"/>
        </lib>
    </war>

    <echo message="[alt] Альтернативная версия собрана:
    ${war.alt.path}"/>
    </target>

    <!-- report — сохранение JUnit-отчёта и git commit (если тесты
    прошли) -->
    <target name="report" depends="test" description="Сохранение
    JUnit-отчёта в Git">
        <!-- Проверяем наличие отчётов -->
        <available file="${dir.test.reports}" type="dir"
        property="reports.exist"/>

```

```

        <antcall target="report.commit"/>
    </target>

    <target name="report.commit" if="reports.exist">
        <mkdir dir="test-reports"/>
        <copy todir="test-reports">
            <fileset dir="${dir.test.reports}" includes="**/*.xml"/>
        </copy>
        <exec executable="${git.executable}" failonerror="true">
            <arg value="add"/>
            <arg value="test-reports"/>
        </exec>
        <exec executable="${git.executable}" failonerror="true">
            <arg value="commit"/>
            <arg value="-m"/>
            <arg value="${git.commit.message}"/>
        </exec>
        <echo message="[report] Отчёт JUnit добавлен в Git и
закоммичен."/>
    </target>

    <!--
        env – сборка и запуск в альтернативных окружениях
    -->
    <target name="env" depends="build" description="Сборка и запуск
в альтернативных окружениях">
        <!-- Окружение 1 -->
        <echo message="[env] Запуск в окружении 1: ${env.1.java}"/>
        <exec executable="${env.1.java}" failonerror="false">
            <arg line="${env.1.jvmargs}"/>
            <arg value="-version"/>
        </exec>

        <!-- Окружение 2 -->
        <echo message="[env] Запуск в окружении 2: ${env.2.java}"/>
        <exec executable="${env.2.java}" failonerror="false">
            <arg line="${env.2.jvmargs}"/>
            <arg value="-version"/>
        </exec>

        <echo message="[env] Проверка окружений завершена."/>
    </target>

```

```

<!--
    diff – проверка SVN-статуса и условный коммит
-->
<target name="diff" description="Проверка SVN-изменений и
условный коммит">
    <!-- Получаем список изменённых файлов -->
    <exec executable="${svn.executable}"
outputproperty="svn.status.output" failonerror="true">
        <arg value="status"/>
    </exec>
    <echo message="[diff] SVN статус:
${line.separator}${svn.status.output}"/>

    <!-- Проверяем, касаются ли изменения исключённых классов
-->
    <condition property="svn.safe.to.commit">
        <not>
            <contains string="${svn.status.output}"
substring="${svn.excluded.classes}"/>
        </not>
    </condition>

    <antcall target="diff.commit"/>
    <antcall target="diff.skip"/>
</target>

<target name="diff.commit" if="svn.safe.to.commit">
<exec executable="${svn.executable}" failonerror="true">
    <arg value="commit"/>
    <arg value="-m"/>
    <arg value="Auto-commit: changes do not affect excluded
classes [Ant]"/>
</exec>
<echo message="[diff] SVN коммит выполнен успешно."/>
</target>

<target name="diff.skip" unless="svn.safe.to.commit">
    <echo message="[diff] Коммит пропущен: изменения
затрагивают исключённые классы (${svn.excluded.classes})."/>
</target>

<!-- history – откат по SVN до последней компилируемой ревизии
-->
<target name="history" description="Поиск последней компилируемой

```

```

SVN-ревизии">
    <exec executable="${svn.executable}"
outputproperty="svn.info.output" failonerror="true">
    <arg value="info"/>
    <arg value="--show-item"/>
    <arg value="revision"/>
</exec>
<echo message="[history] Текущая ревизия: ${svn.info.output}"/>

<mkdir dir="${dir.build}"/>

<!-- Получаем XML лог SVN -->
<exec executable="${svn.executable}" output="${dir.build}/svn-
log.xml" failonerror="true">
    <arg value="log"/>
    <arg value="--xml"/>
    <arg value="-q"/>
</exec>

<!-- Вытаскиваем номера ревизий через xpath в файл -->
<xslt in="${dir.build}/svn-log.xml"
out="${dir.build}/svn-revisions.txt"
style="revisions.xsl"/>

<loadfile property="svn.rev.list" srcFile="${dir.build}/svn-
revisions.txt"/>
<echo message="[history] Список ревизий: ${svn.rev.list}"/>

<var name="history.found" value="false"/>
<var name="history.good.rev" value="0"/>

<for param="cur.rev" list="${svn.rev.list}"
delimiter="${line.separator}">
    <sequential>
        <if>
            <equals arg1="${history.found}" arg2="false"/>
            <then>
                <echo message="[history] Пробуем ревизию
@{cur.rev}...">

                <exec executable="${svn.executable}"
failonerror="false">
                    <arg value="update"/>
                    <arg value="-r"/>

```

```

        <arg value="@{cur.rev}"/>
    </exec>

    <exec executable="${svn.executable}"
failonerror="false">
        <arg value="resolve"/>
        <arg value="--accept"/>
        <arg value="working"/>
        <arg value="--depth"/>
        <arg value="infinity"/>
        <arg value="."/>
    </exec>

    <delete dir="${dir.classes}" quiet="true"/>
    <mkdir dir="${dir.classes}"/>

    <trycatch>
        <try>
            <javac
                srcdir="${dir.src}"
                destdir="${dir.classes}"
                source="${javac.source}"
                target="${javac.target}"
                encoding="${javac.encoding}"
                includeantruntime="false"
                failonerror="true">
                <classpath refid="compile.classpath"/>
            </javac>
            <echo message="[history] Компиляция успешна на
реvisions @{cur.rev}!"/>
            <var name="history.found" value="true"/>
            <var name="history.good.rev" value="@{cur.rev}"/>
        </try>
        <catch>
            <echo message="[history] Ревизия @{cur.rev} не
компилируется..."/>
            <delete dir="${dir.classes}" quiet="true"/>
        </catch>
    </trycatch>
</then>
</if>
</sequential>
</for>

```

```

<if>
  <not><equals arg1="${history.found}" arg2="false"/></not>
  <then>
    <echo message="[history] Последняя рабочая ревизия:
${history.good.rev}"/>
    <exec executable="${svn.executable}"
      outputproperty="svn.diff.output"
      failonerror="false">
      <arg value="diff"/>
      <arg value="-r"/>
      <arg value="${history.good.rev}:HEAD"/>
    </exec>
    <mkdir dir="${dir.build}"/>
    <echo file="${dir.build}/last-working-diff.txt"
message="${svn.diff.output}"/>
    <echo message="[history] Diff сохранён в ${dir.build}/last-
working-diff.txt"/>
  </then>
  <else>
    <echo message="[history] Ни одна ревизия не
компилируется!"/>
  </else>
</if>
</target>
<!-- team – получение 4 предыдущих git-ревизий, сборка и упаковка
-->
  <target name="team" description="Сборка 4 предыдущих git-
ревизий и упаковка в ZIP">
    <mkdir dir="${dir.dist}/team-builds"/>

    <!-- Получаем хеши последних N коммитов -->
    <exec executable="${git.executable}"
outputproperty="git.log.output" failonerror="true">
      <arg value="log"/>
      <arg value="--pretty=format:%H"/>
      <arg value="--${team.revisions.count}"/>
    </exec>

    <echo message="[team] Ревизии для сборки:
${line.separator}${git.log.output}"/>

    <!-- Перебираем список ревизий -->
    <for list="${git.log.output}" delimiter="${line.separator}"
param="rev">

```

```

<sequential>
  <propertyregex property="trimmed.rev"
    input="@{rev}"
    regexp="^[ \t]*(.*?)[ \t]*$"
    select="\1"
    override="true"/>
  <if>
    <not><equals arg1="${trimmed.rev}" arg2=""/></not>
    <then>
      <propertyregex property="short.rev"
        input="${trimmed.rev}"
        regexp="^(.{1,7})"
        select="\1"
        override="true"/>

      <exec executable="${git.executable}"
failonerror="false">
        <arg value="checkout"/>
        <arg value="${trimmed.rev}"/>
      </exec>

      <antcall target="build"/>

      <copy file="${war.path}"
        tofile="${dir.dist}/team-builds/
${project.name}-${short.rev}.war"
        overwrite="true"/>

      <echo message="[team] Ревизия ${short.rev}
собрана."/>
    </then>
  </if>
</sequential>
</for>

<exec executable="${git.executable}" failonerror="false">
  <arg value="checkout"/>
  <arg value="${git.branch}"/>
</exec>

<!-- Упаковываем все WAR в ZIP -->
<zip destfile="${zip.team.path}">
  <fileset dir="${dir.dist}/team-builds" includes="**/
*.war"/>

```

```
        </zip>

        <echo message="[team] Все ревизии упакованы в:
${zip.team.path}"/>
    </target>
</project>
```

build.properties:

```
# Метаданные проекта
project.name=WebLab3
project.version=1.0.0
project.vendor=Roman P.

# Структура директорий
dir.src=src/main/java
dir.resources=src/main/resources
dir.webapp=src/main/webapp
dir.build=build
dir.classes=${dir.build}/classes
dir.dist=dist
dir.lib=lib
dir.test.src=src/test/java
dir.test.classes=${dir.build}/test-classes
dir.test.reports=${dir.build}/test-reports
dir.javadoc=${dir.build}/javadoc
dir.alt=alt
dir.alt.src=${dir.alt}/src
dir.alt.classes=${dir.alt}/classes
dir.alt.dist=${dir.alt}/dist
dir.il8n.ascii=${dir.build}/il8n-ascii

# Артефакты
war.name=${project.name}-${project.version}.war
war.path=${dir.dist}/${war.name}
war.alt.name=${project.name}-alt-${project.version}.war
war.alt.path=${dir.alt.dist}/${war.alt.name}
zip.team.name=${project.name}-team-builds.zip
zip.team.path=${dir.dist}/${zip.team.name}

# Компиляция
javac.source=17
```

```
javac.target=17
javac.encoding=UTF-8
javac.debug=true

# Зависимости
lib.compile=${dir.lib}/compile
lib.runtime=${dir.lib}/runtime
lib.test=lib/test

# Манифест
manifest.vendor=${project.vendor}
manifest.title=${project.name}
manifest.version=${project.version}

# JUnit
junit.haltonerror=false
junit.haltonfailure=false
junit.fork=true

# Git
git.executable=git
git.branch=master
git.commit.message=Auto-commit: JUnit report [Ant]

# SVN
svn.executable=svn
svn.url=file:///C:/svn-repos/weblab3/trunk
# Классы, изменения которых блокируют SVN-коммит (через пробел)
svn.excluded.classes=PointLogicService

# Музыка
music.file=resources/success.wav

# SCP
scp.host=se.ifmo.ru
scp.port=2222
scp.user=s467082
scp.remotedir=/home/studs/s467082
scp.password=secret123

# alt: замена имён (replaceregexp)
alt.replace.from.class=InputBean
alt.replace.to.class=FormBean
alt.replace.from.var=historyService
```

```
alt.replace.to.var=svcHistory
```

```
# env: альтернативные окружения
```

```
env.1.java=C:/Program Files/Java/jdk-17/bin/java
```

```
env.1.jvmargs=-Xmx256m -Dfile.encoding=UTF-8
```

```
env.2.java=C:/Program Files/Java/jdk-17/bin/java
```

```
env.2.jvmargs=-Xmx512m -Dfile.encoding=UTF-8
```

```
# team: количество предыдущих git-ревизий
```

```
team.revisions.count=4
```

3. Заключение

при выполнении лабораторной работы я познакомился с системой сборки Apache Ant и научился создавать на ней задачи для исполнения.